COMPUTER ENGINEERING

QUEZON CITY

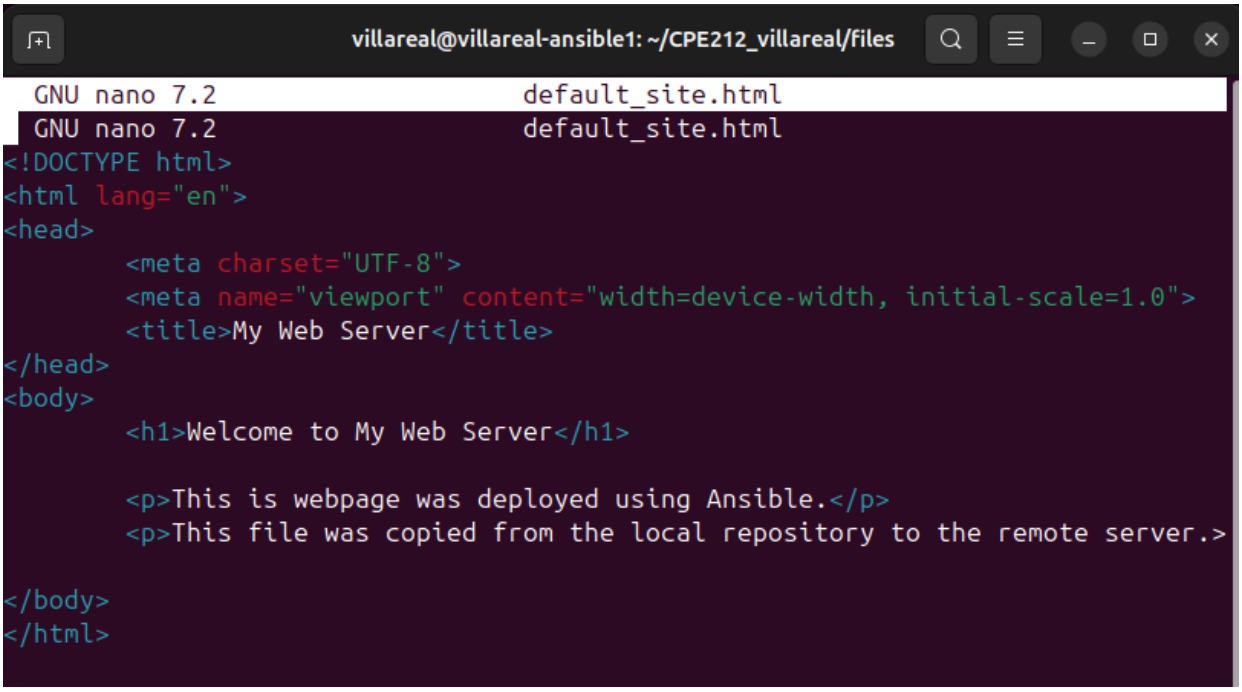
COURSE CODE/TITLE:	CPE 212 | Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	9/10/26	**DATE SUBMITTED:**	9/10/26
NAME:	Villareal, Vanderohe Vernon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7 **Managing Files and Creating Roles in Ansible**

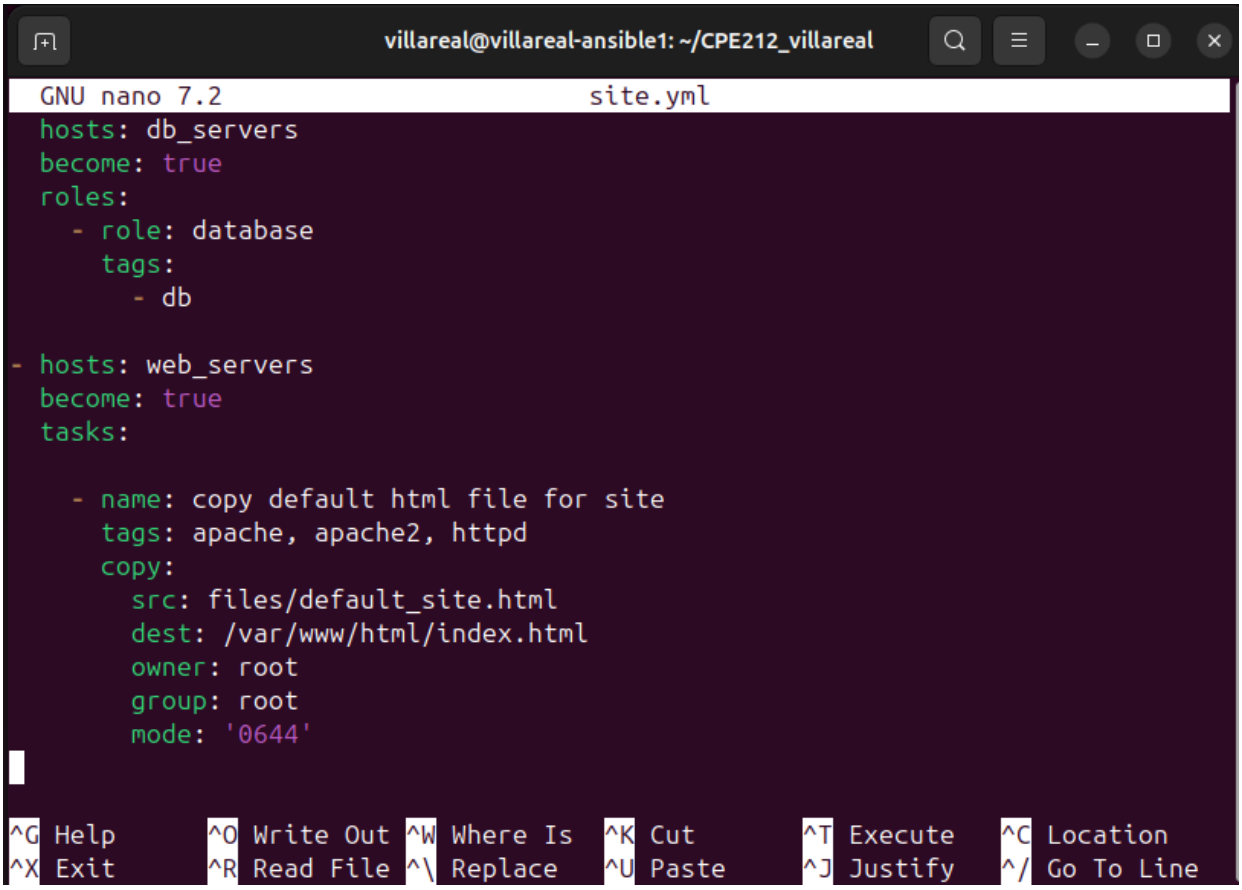
1. PROCEDURE OUTPUT

Task 1:

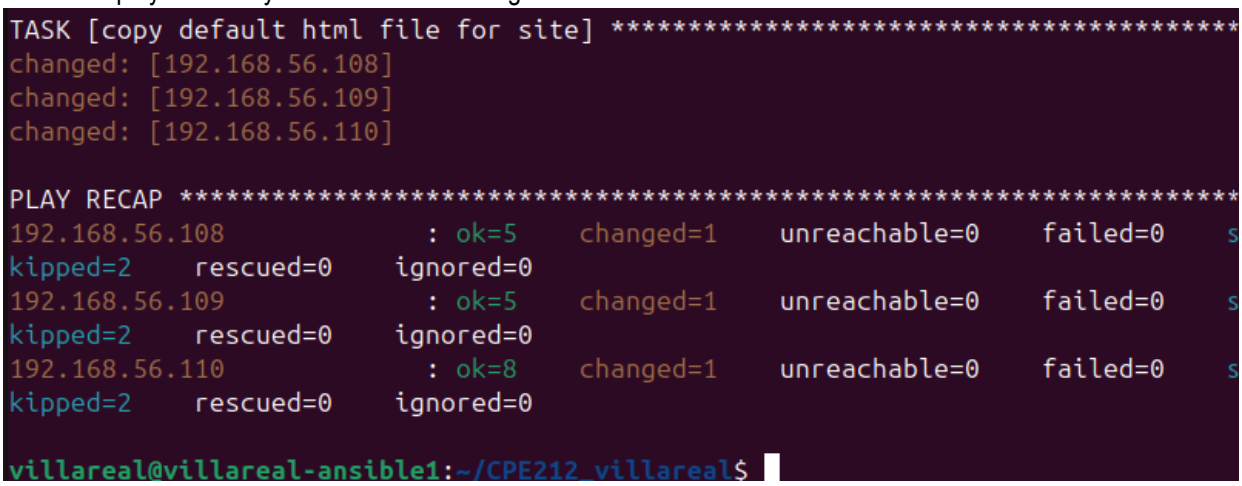
1. Using the previous directory that was created in your previous laboratory, create a directory, and name it “files.” Create a file inside that directory and name it “default_site.html.” Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit.



2. Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site:



3. Run the playbook site.yml. Describe the changes.



- There are changes to the web servers.

4. Go to the remote servers (web_servers) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (default_site.html). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.

Managed Node 1:

```
villareal@villareal-ansible1:~$ cat /var/www/html/index.html
  GNU nano 7.2                    default_site.html
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Web Server</title>
</head>
<body>
        <h1>Welcome to My Web Server</h1>

        <p>This is webpage was deployed using Ansible.</p>
        <p>This file was copied from the local repository to the remote server.

</body>
</html>
```

Managed Node 2:

```
villareal@villareal-ansible1:~$ cat /var/www/html/index.html
  GNU nano 7.2                    default_site.html
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Web Server</title>
</head>
<body>
        <h1>Welcome to My Web Server</h1>

        <p>This is webpage was deployed using Ansible.</p>
        <p>This file was copied from the local repository to the remote server.>

</body>
</html>
```

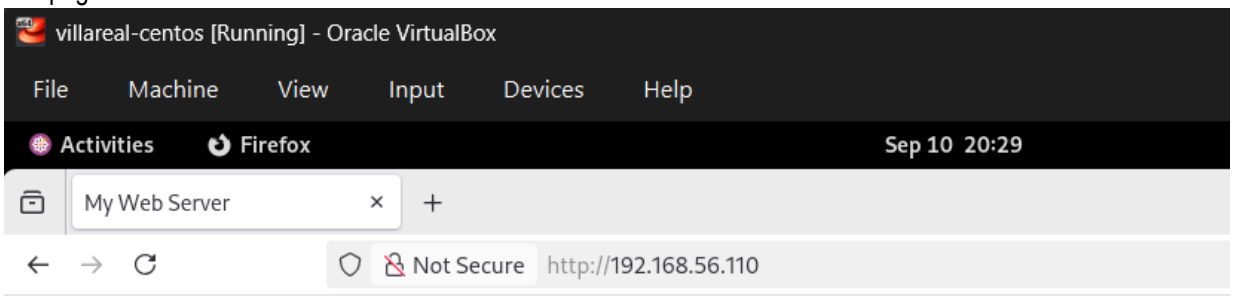
Managed Node 3 (CentOS):

```
[villareal@localhost ~]$ cat /var/www/html/index.html
  GNU nano 7.2                    default_site.html
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>My Web Server</title>
</head>
<body>
        <h1>Welcome to My Web Server</h1>

        <p>This is webpage was deployed using Ansible.</p>
        <p>This file was copied from the local repository to the remote server.></p>

</body>
</html>
```

Webpage:



GNU nano 7.2 default_site.html

Welcome to My Web Server

This is webpage was deployed using Ansible.

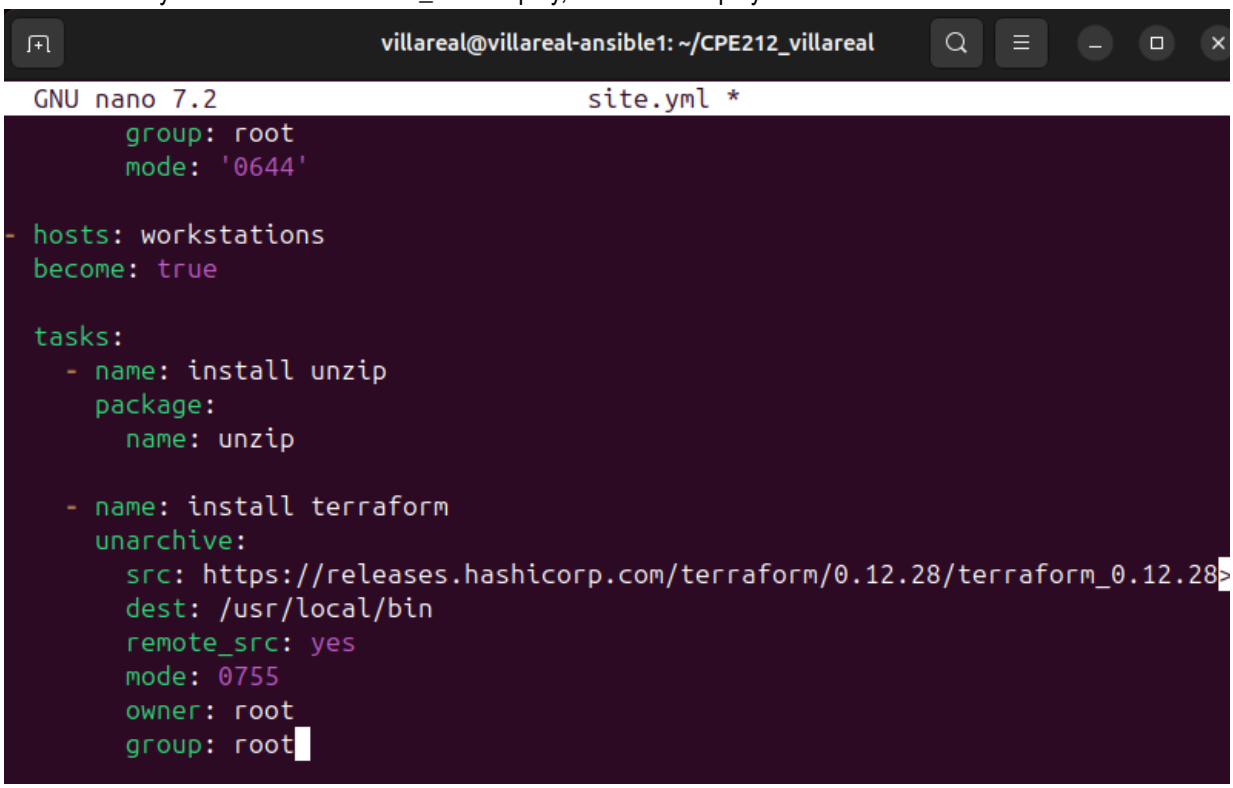
This file was copied from the local repository to the remote server.>

5. Sync your local repository with GitHub and describe the changes.

```
villareal@villareal-ansible1:~/CPE212_villareal$ git add .
villareal@villareal-ansible1:~/CPE212_villareal$ git commit -m "HOA7 TASK 1"
[main 27845cd] HOA7 TASK 1
 2 files changed, 29 insertions(+)
 create mode 100644 files/default_site.html
villareal@villareal-ansible1:~/CPE212_villareal$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 869 bytes | 869.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:VanderoheVillareal/CPE212_villareal.git
   7549927..27845cd  main -> main
villareal@villareal-ansible1:~/CPE212_villareal$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

Task 2:

1. Edit the site.yml. Just before the web_servers play, create a new play:



2. Edit the inventory file and add a workstation group. Add any Ubuntu remote server. Make sure to remember the IP address.

```
villareal@villareal-ansible1: ~/CPE212_villareal
  GNU nano 7.2                                  inventory
[web_servers]
192.168.56.108
192.168.56.109
192.168.56.110

[db_servers]
192.168.56.110

[file_servers]
192.168.56.110

[workstations]
192.168.56.109
```

3. Run the playbook. Describe the output

```
TASK [install terraform] ********************************************************
changed: [192.168.56.109]

PLAY RECAP **********************************************************************
192.168.56.108             : ok=5    changed=0    unreachable=0    failed=0
kipped=2    rescued=0    ignored=0
192.168.56.109             : ok=8    changed=1    unreachable=0    failed=0
kipped=2    rescued=0    ignored=0
192.168.56.110             : ok=8    changed=0    unreachable=0    failed=0
kipped=2    rescued=0    ignored=0
```

- The terraform is now installed on the remote server.

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

```
villareal@villareal-ansible1:~$ terraform
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.

Common commands:
    apply              Builds or changes infrastructure
    console            Interactive console for Terraform interpolations
    destroy            Destroy Terraform-managed infrastructure
    env                Workspace management
    fmt                Rewrites config files to canonical format
    get                Download and install modules for the configuration
    graph              Create a visual graph of Terraform resources
    import             Import existing infrastructure into Terraform
    init               Initialize a Terraform working directory
    login              Obtain and save credentials for a remote host
    logout             Remove locally-stored credentials for a remote host
```

- By using the command terraform it outputs commands that can be used when terraform is installed.

Task 3:

1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

```
  GNU nano 7.2                                site2.yml
---
- hosts: all
  become: true
  pre_tasks:

  - name: update repository index (CentOS)
    tags: always
    dnf:
      update_cache: yes
    changed_when: false
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      update_cache: yes
    changed_when: false
    when: ansbile_distribution == "Ubuntu"

- hosts: all
                                [ Read 43 lines ]
```

```
- hosts: workstations
  become: true
  roles:
    - workstations

- hosts: web_servers
  become: true
  roles:
    - web_servers

- hosts: db_servers
  become: true
  roles:
    - db_servers

- hosts: file_servers
  become: true
  roles:
    - file_serves
```

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.

```
villareal@villareal-ansible1:~/CPE212_villareal/roles$ mkdir base
villareal@villareal-ansible1:~/CPE212_villareal/roles$ mkdir web_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles$ mkdir file_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles$ mkdir db_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles$ mkdir workstations
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd base
villareal@villareal-ansible1:~/CPE212_villareal/roles/base$ mkdir tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/base$ cd -
/home/villareal/CPE212_villareal/roles
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd web_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles/web_servers$ mkdir tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/web_servers$ cd -
/home/villareal/CPE212_villareal/roles
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd file_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles/file_servers$ mkdir tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/file_servers$ cd -
/home/villareal/CPE212_villareal/roles
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd db_servers
villareal@villareal-ansible1:~/CPE212_villareal/roles/db_servers$ mkdir tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/db_servers$ cd -
/home/villareal/CPE212_villareal/roles
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd workstations
villareal@villareal-ansible1:~/CPE212_villareal/roles/workstations$ mkdir tasks
```

3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.

```
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd workstations/tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/workstations/tasks$ touch
main.yml
villareal@villareal-ansible1:~/CPE212_villareal/roles/workstations/tasks$ cd ../
..
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd db_servers/tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/db_servers/tasks$ touch ma
in.yml
villareal@villareal-ansible1:~/CPE212_villareal/roles/db_servers/tasks$ cd ../..
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd file_servers/tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/file_servers/tasks$ touch
main.yml
villareal@villareal-ansible1:~/CPE212_villareal/roles/file_servers/tasks$ cd ../
..
villareal@villareal-ansible1:~/CPE212_villareal/roles$ cd web_servers/tasks
villareal@villareal-ansible1:~/CPE212_villareal/roles/web_servers/tasks$ touch m
ain.yml
```

web_servers:

```
GNU nano 7.2                          main.yml *
- hosts: web_servers
  become: true
  tasks:

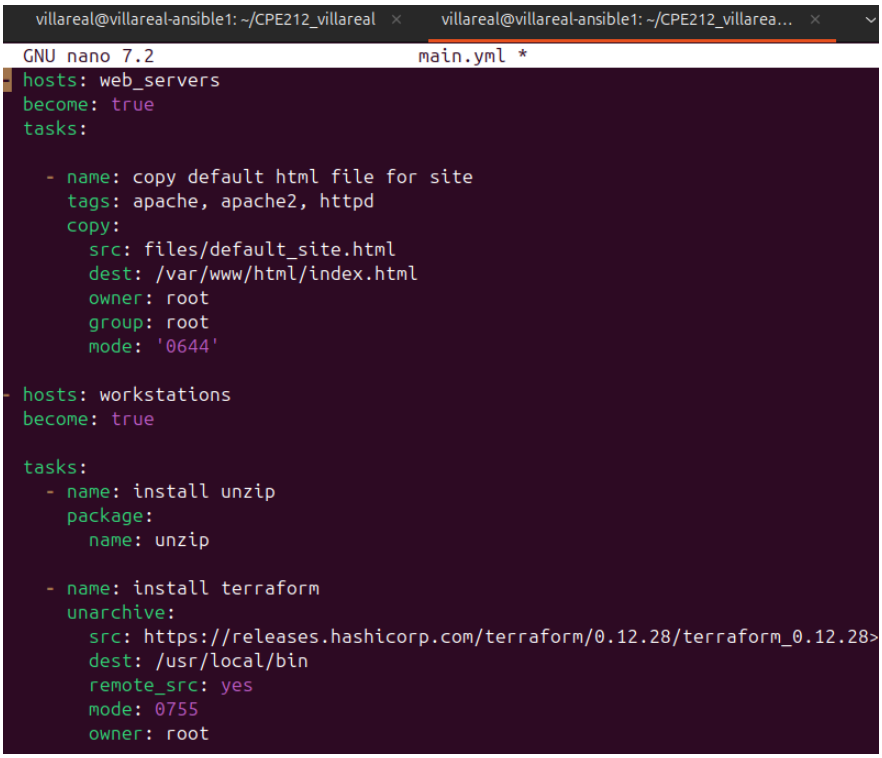
    - name: copy default html file for site
      tags: apache, apache2, httpd
      copy:
        src: files/default_site.html
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: '0644'

- hosts: workstations
  become: true

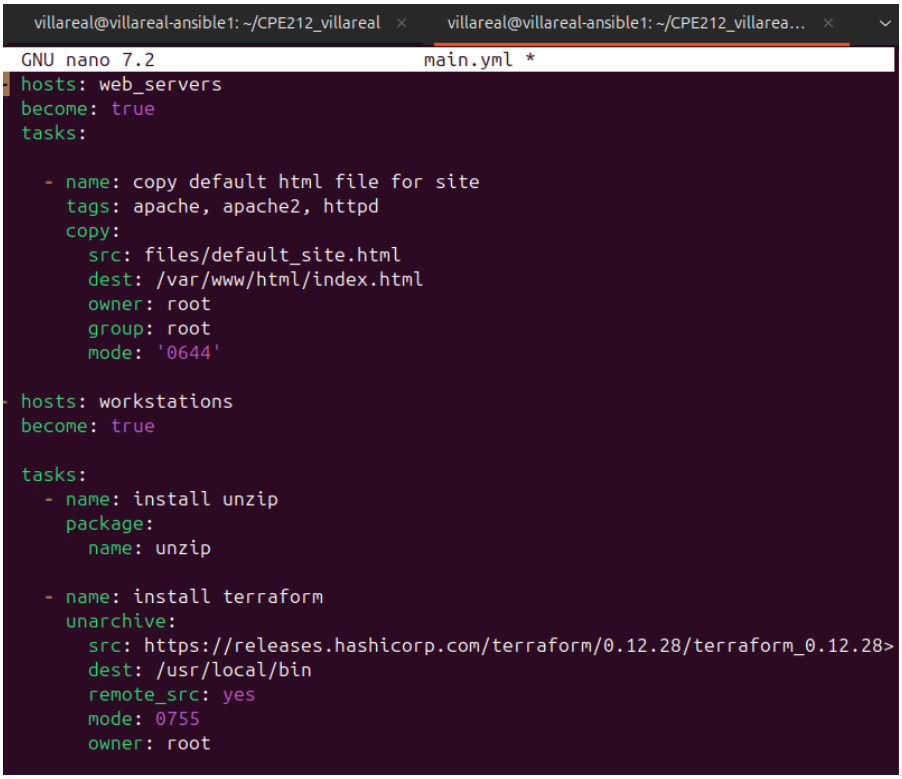
  tasks:
    - name: install unzip
      package:
        name: unzip

    - name: install terraform
      unarchive:
        src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28>
        dest: /usr/local/bin
        remote_src: yes
        mode: 0755
        owner: root
```

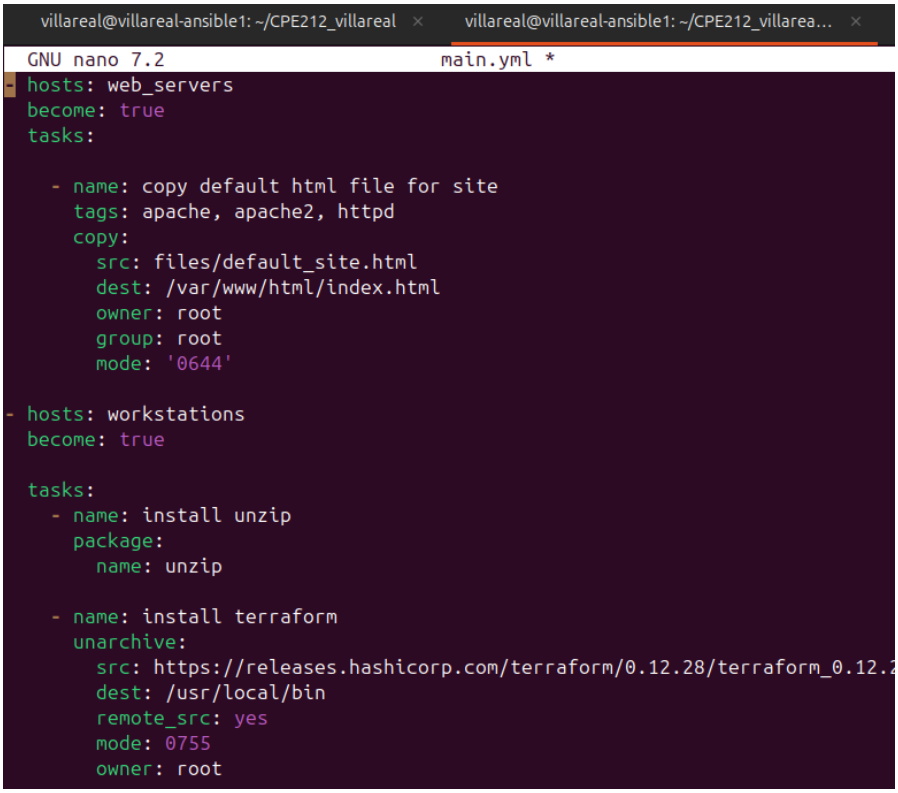
file_servers:



db_servers:



workstations:



4. Run the site.yml playbook and describe the output.

```
PLAY RECAP *********************************************************************
**
192.168.56.108             : ok=6    changed=1    unreachable=0    failed=0
 skipped=1    rescued=0    ignored=0
192.168.56.109             : ok=9    changed=0    unreachable=0    failed=0
 skipped=1    rescued=0    ignored=0
192.168.56.110             : ok=12   changed=1    unreachable=0    failed=0
 skipped=1    rescued=0    ignored=0
```

- The playbook ran successfully, confirming that the Ansible roles were correctly applied to the server groups.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of creating roles?
   - **Creating roles makes Ansible projects more organized, reusable, and easier to maintain. Each role contains tasks for a specific purpose, making it easier to manage different types of servers and reuse the same configuration.**

2. What is the importance of managing files?

- **File management is important because administrators often need to create, copy, modify, and control files on multiple remote servers. Ansible allows these operations to be automated and performed consistently without manually accessing every server.**

Conclusions/Learnings:
Through this hands-on-activity, I learned how Ansible can be used to manage files and automate server configuration. I learned how to copy files to remote servers, install software using Ansible modules, and organize tasks using roles. Using roles makes the project easier to organize, reuse, and maintain across multiple servers.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.